

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA0603

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I D. Sindhu (192424197) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

D. Sindhu.

Signature of the student:

Annexure A

Short Answer Test

1. Compare K-Nearest Neighbour and Case-Based Learning using a real-world example such as medical diagnosis. Analyze differences in memory usage, similarity measures, and prediction strategy.
2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.
3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.
4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.
5. Compare Decision Tree Learning and K-Nearest Neighbour (KNN) using a classification problem such as customer churn prediction. Analyze differences in model interpretability, training time, handling of noisy data, and decision-making process. Discuss how each method performs with large datasets and complex feature spaces.
6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.
7. Analyze the differences between Support Vector Machines (SVM) and KNN in classification tasks. Use an example dataset to compare decision boundaries, scalability, sensitivity to noise, and computational complexity. Discuss how kernel functions in SVM improve performance for non-linear problems.
8. Compare K-Means Clustering and Hierarchical Clustering using a dataset with unknown structure. Analyze differences in cluster formation, computational cost, scalability, and interpretability. Discuss how each method handles varying cluster shapes and the impact of choosing the number of clusters.
9. Compare Genetic Algorithms and Gradient Descent Optimization for solving optimization problems. Analyze differences in convergence behavior, ability to escape local minima, computational cost, and suitability for different types of objective functions. Provide examples where each method is more effective.

10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

11. Compare **Random Forest and Decision Tree** algorithms using a classification problem such as credit risk assessment. Analyze differences in overfitting control, accuracy, interpretability, and robustness to noisy data. Discuss how ensemble learning in Random Forest improves generalization compared to a single decision tree.

12. Compare **Linear Regression and Polynomial Regression** for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.

13. Compare **K-Means Clustering and DBSCAN** using a dataset with irregular cluster shapes. Analyze differences in cluster detection, handling of noise and outliers, requirement of parameters, and scalability. Discuss how each algorithm performs in real-world clustering tasks.

14. Compare **Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM)** networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.

15. Compare **Naïve Bayes and Support Vector Machines (SVM)** for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

1. KNN vs Case-Based Learning – Medical Diagnosis

KNN:

- Stores the training data.
- Finds the **K nearest patients** based on similarity.
- Uses distance measures such as Euclidean distance.
- Predicts the diagnosis using the majority class.
- Mainly used for classification.

Case-Based Learning:

- Stores previous cases along with their solutions.
- Finds the most similar previous patient case.
- Reuses the solution of the similar case.
- New cases can be added to the case database.
- More suitable when previous complete cases are available.

Conclusion: KNN predicts using nearby neighbours, while Case-Based Learning solves a new problem by retrieving and reusing a similar past case.

2. LWR vs RBF – Function Approximation

Locally Weighted Regression:

- Gives higher weight to data points closer to the query.
- Gives lower weight to distant points.
- Performs regression locally.
- Computational cost can be high because weights are calculated for each query.
- Produces smooth local predictions.

RBF Network:

- Uses Gaussian radial basis functions.
- Each RBF has a centre and width σ .
- Learns the weights of the basis functions.
- Prediction is generally faster after training.
- Produces smooth predictions.

Conclusion: LWR focuses strongly on the current query point, while RBF learns basis functions that can be reused for many predictions.

3. KNN vs RBF – Classification**KNN:**

- Requires very little training.
- Stores the training examples.
- Uses nearby points to classify a new point.
- Decision boundary can be irregular.
- Prediction becomes slower with large datasets.

RBF:

- Requires training to learn weights.
- Uses Gaussian/RBF activation functions.
- Produces smoother decision boundaries.
- Usually gives faster prediction after training.
- Can generalize well when centres and σ are properly selected.

Conclusion: KNN is simple and easy to implement, while RBF requires more training but can produce smoother decision boundaries.

4. Case-Based Learning vs LWR – Recommendation System**Case-Based Learning:**

- Stores previous customer cases.
- Finds customers with similar preferences.
- Recommends products based on similar previous cases.
- New cases can easily be added.
- Useful when previous solutions are available.

Locally Weighted Regression:

- Uses nearby data points for prediction.
- Gives more importance to similar data points.
- Can predict a customer's rating or preference.
- Requires calculations for every query.
- Suitable for continuous numerical predictions.

Conclusion: Case-Based Learning is useful for reusing previous recommendations, while LWR is useful for predicting numerical preferences.

5. Decision Tree vs KNN – Customer Churn**Decision Tree:**

- Uses if-else decision rules.
- Easy to understand and interpret.
- Training is required before prediction.
- Can handle nonlinear relationships.
- Can become overfitted when the tree is too deep.
- Generally, works well with large datasets.

KNN:

- Stores training data.
- Finds customers similar to the new customer.
- Uses distance to make predictions.
- Requires little training.
- Prediction can be slow for large datasets.
- Sensitive to feature scaling and noisy data.

Conclusion: Decision Trees are more interpretable and efficient for large datasets, while KNN is simpler but can be expensive during prediction.

6. Naïve Bayes vs Logistic Regression – Spam Detection**Naïve Bayes:**

- Based on probability.
- Assumes features are conditionally independent.
- Very fast for training and prediction.
- Works well with high-dimensional text data.
- Performs well even with relatively small datasets.

Logistic Regression:

- Uses the sigmoid function.
- Does not require the same feature-independence assumption.
- Predicts the probability of a class.
- Works well with high-dimensional data.
- Requires optimization during training.

Conclusion: Naïve Bayes is very fast and effective for text classification, while Logistic Regression can perform better when there are enough training data and useful feature relationships.

7. SVM vs KNN**SVM:**

- Finds an optimal decision boundary.
- Maximizes the margin between classes.
- Requires training.
- Prediction is usually fast after training.
- Can handle nonlinear problems using kernel functions.
- RBF kernel can create nonlinear decision boundaries.

KNN:

- Does not build a complex model during training.
- Uses nearby training points for classification.
- Prediction becomes expensive for large datasets.
- Sensitive to noisy and irrelevant features.
- Can naturally produce nonlinear boundaries.

Conclusion: SVM is generally better for large classification tasks requiring a strong decision boundary, while KNN is simpler and useful for smaller datasets.

8. K-Means vs Hierarchical Clustering**K-Means:**

- Divides data into K clusters.
- Requires the number of clusters beforehand.
- Uses centroids.
- Faster and more scalable.

- Works best with roughly spherical clusters.
- Simple to understand.

Hierarchical Clustering:

- Builds a hierarchy of clusters.
- Does not require K at the beginning.
- Produces a dendrogram.
- Usually more computationally expensive.
- Useful for understanding relationships between clusters.
- Different linkage methods can produce different clusters.

Conclusion: K-Means is faster and suitable for large datasets, while Hierarchical Clustering provides a detailed view of cluster relationships.

9. Genetic Algorithm vs Gradient Descent

Genetic Algorithm:

- Uses a population of possible solutions.
- Uses selection, crossover and mutation.
- Does not require derivatives.
- Can escape local minima more easily.
- Computationally expensive.
- Useful for complex or non-differentiable problems.

Gradient Descent:

- Starts with a solution and improves it step by step.
- Uses the gradient of the objective function.
- Usually converges faster.
- Can get stuck in local minima.
- Computationally less expensive.
- Useful for differentiable functions such as neural-network training.

Conclusion: Genetic Algorithms are suitable for complex search problems, while Gradient Descent is faster for smooth and differentiable optimization problems.

10. PCA vs Feature Selection

PCA:

- Creates new features called principal components.
- Reduces the number of dimensions.
- Tries to preserve maximum variance.
- Original feature meaning may be lost.
- Useful for high-dimensional datasets.
- Can reduce noise and improve model efficiency.

Feature Selection:

- Selects important features from the original dataset.
- Does not create new features.
- Keeps the original meaning of features.
- Easier to interpret.
- Can remove irrelevant features.
- Usually simpler than PCA.

11. Random Forest vs Decision Tree – Credit Risk Assessment

Decision Tree

- Uses a single tree to make decisions.
- Easy to understand and interpret.

- Can easily overfit the training data.
- Sensitive to noisy data.
- Training and prediction are relatively simple.
- Example: A tree can decide credit risk using income, credit score, and loan amount.

Random Forest

- Combines many decision trees.
- Uses **ensemble learning** to make the final prediction.
- Reduces overfitting compared with a single tree.
- More robust to noisy data.
- Usually provides higher accuracy.
- Less interpretable than a single Decision Tree.
- Random Forest combines the predictions of many trees, usually by majority voting for classification.

Conclusion

Random Forest generally provides **better generalization and accuracy** because many trees work together. A Decision Tree is easier to understand, while Random Forest is usually more robust and less prone to overfitting.

12. Linear Regression vs Polynomial Regression – Housing Prices

Linear Regression

- Assumes a linear relationship between input and output.
- Simple and easy to understand.
- Has lower model complexity.
- Less likely to overfit.
- Works well when housing prices change approximately linearly.
- Example: Price increases steadily as house size increases.

Polynomial Regression

- Adds polynomial terms such as x^2 , x^3 , etc.
- Can capture **non-linear relationships**.
- More complex than Linear Regression.
- Higher risk of overfitting, especially with high polynomial degrees.
- Can provide better predictions when the relationship is curved.
- Less interpretable than a simple linear model.

Conclusion

Linear Regression is better for simple linear relationships, while Polynomial Regression is better when housing prices have a **non-linear relationship** with features such as area or age.

13. K-Means vs DBSCAN – Irregular Cluster Shapes

K-Means

- Divides data into a specified number of clusters.
- Requires the value of **K** beforehand.
- Uses centroids to form clusters.
- Works best for roughly spherical and similarly sized clusters.
- Sensitive to outliers and noise.
- Usually faster and scalable for large datasets.
- May perform poorly with irregularly shaped clusters.

DBSCAN

- Groups points based on **density**.
- Does not require the number of clusters beforehand.
- Can detect irregularly shaped clusters.

- Can identify noise and outliers.
- Requires parameters such as eps and min_samples.
- Can be slower than K-Means for some large datasets.
- Useful for real-world data containing noise and irregular patterns.

Conclusion

K-Means is simple and fast but works poorly with irregular shapes. **DBSCAN is better for irregular clusters and noisy data** because it is density-based.

14. RNN vs LSTM – Stock Price Prediction

RNN

- Designed for sequential and time-series data.
- Uses previous information to predict future values.
- Simple compared with LSTM.
- Can have difficulty remembering information from far back in the sequence.
- Suffers from the **vanishing gradient problem**.
- Has lower computational complexity than LSTM.

LSTM

- A special type of RNN.
- Designed to handle **long-term dependencies**.
- Uses gates to control what information is remembered or forgotten.
- Reduces the vanishing gradient problem.
- Can provide better performance for long sequences.
- More computationally complex than a basic RNN.

Conclusion

RNN is simpler and faster but may struggle with long-term dependencies. **LSTM is more suitable for long time-series data** because it can remember important information for longer periods.

15. Naïve Bayes vs SVM – Sentiment Analysis

Naïve Bayes

- Based on probability.
- Assumes features are conditionally independent.
- Very fast for training and prediction.
- Works well with high-dimensional text data.
- Simple to implement.
- Suitable for large text datasets.
- Often works well even with limited training data.

SVM

- Finds a decision boundary that maximizes the margin between classes.
- Does not make the same feature-independence assumption as Naïve Bayes.
- Can create complex decision boundaries.
- Usually requires more computational effort than Naïve Bayes.
- Works well with high-dimensional data.
- Can use kernels for non-linear classification.